

第三章：多道程序与分时多任务

引言

本章导读

本章将介绍操作系统的几项关键功能：

1. 通过提前将应用程序加载到内存，减少应用切换时的开销。
2. 通过协作机制，支持程序主动让出处理器，从而提高系统执行效率。
3. 通过抢占机制，支持程序被动让出处理器，提升不同程序之间使用处理器资源的公平性，同时也增强了应用对外设输入输出事件的响应效率。

上一章中，我们实现了一个简单的批处理系统。它主要具备两个能力：首先，系统能够按顺序自动加载并运行应用序列中的每一个程序，当一个程序运行结束后，无需操作员手动替换；其次，借助硬件提供的特权级机制，运行在更高特权级的批处理系统不会受到用户态应用程序的有意或无意错误的影响。它可以全面监控运行在用户态特权级下的应用执行，一旦应用越过硬件设置的特权级界限，或者主动请求操作系统服务，就会触发异常陷入，进入批处理系统进行处理。无论是因为应用出错，还是应用主动声明执行完毕，批处理系统都只需加载序列中的下一个应用并启动执行。由此可见，批处理系统的特点是：在内存中同一时间最多只需驻留一个应用程序。这是因为只有当前应用出错或退出后，批处理系统才会将下一个应用加载到同一块内存区域。

随着计算机硬件的快速发展，内存容量逐步增加，处理器速度也在不断提升，但外设输入输出性能的提升相对有限。这使得以往内存只能容纳一个程序的情况得到明显改善，同时处理器的空闲时间也增加了。于是，研究者开始考虑在内存中尽量同时驻留多个应用程序，以提高处理器的利用率。不过，在这种方式下，仍然需要等到一个程序执行完毕或主动让出执行权，处理器才能切换到另一个程序。这种运行模式被称为“多道程序”。

协作式操作系统

早期的计算机系统大多采用单处理器架构。随着处理器性能的持续提升，其与输入输出设备的速度差距也越来越大。此时，计算机科学家们发现，在多道程序的运行模式下，如果一个程序一直占用处理器而不主动让出，其他程序将无法获得执行机会。如果某个应用程序由于执行输入输出操作而导致处理器空闲，或者让处理器处于忙等状态，那么其他需要利用处理器进行计算的任务，仍然无法使用这部分空闲的处理器资源。

因此，人们提出了一种思路：让应用程序在执行输入输出操作时，能够主动释放处理器，使得其他应用程序可以继续执行。当然，执行放弃处理器这一操作，本质上也属于对处理器资源的一种直接管理。因此，应用程序可以通过发出系统调用的方式，将这一请求提交给操作系统，由操作系统具体完成处理器切换的工作。能够支持这种机制的操作系统，就被称为支持多道程序的协作式操作系统。

抢占式操作系统

计算机科学家很快发现，编写应用程序的科学家（简称应用程序员）通常来自不同领域，他们不一定具备友好协作的意识，也不了解其他程序的运行情况，因此很难——也没有必要——站在提高整个系统利用率的高度去编程。在他们看来，整个计算机似乎就是为运行他们自己的程序而准备的，不需要考虑其他程序如何执行。这就导致应用程序员在编写程序时，很难在合适的位置主动插入“放弃处理器的系统调用请求”，从而导致系统整体利用率依然无法提升。

因此，从系统的角度来看，仍然需要一种能够强制打断应用程序执行的机制，以提高整体系统的效率，并让多个程序在共享计算机资源时相对公平。根据计算机硬件设计，为了提高输入输出效率，外部设备可以通过硬件中断机制与处理器进行交互。这种硬件中断机制能够随时打断正在执行的应用程序，并让操作系统接管，以响应外部设备的输入输出请求。

操作系统还可以进一步利用某种周期性触发的外部设备中断（例如时钟中断），来强制打断某个程序的执行。这样，一个程序只能运行一段固定的时间（这段时间可以简称为一个“时间片”，Time Slice），之后就必须让出处理器。操作系统在处理完外部设备的中断响应后，可以让不同的应用程序轮流使用处理器执行。同时，操作系统可以通过统计程序占用处理器的总时间，来评估该程序对处理器资源的消耗。

我们可以把一个程序在一个时间片内占用处理器执行的过程称为一个“任务”（Task）。让操作系统来管理不同程序的这些任务，通过平衡各程序在整体时间段内获得的任务数量，就能在一定程度上实现系统资源的公平分配和整体效率的提升。在一段包含多个时间片的时间内，会有多个属于不同程序的任务轮流占用处理器执行，这样的操作系统就是支持“分时多任务”的抢占式操作系统。

本章所介绍的多道程序和分时多任务系统都具有一些共同特点：在内存中，同一时间可以驻留多个应用程序。所有应用都是在系统启动时分别加载到内存的不同区域中。由于当前计算机系统通常只有一个处理器，因此同一时间最多只能有一个应用在执行，其余应用则处于就绪状态，需要内核将处理器分配给它们才能开始执行。一旦应用开始执行，它就进入运行状态。

本章我们主要关注抢占式的、支持多进程的操作系统，通过时钟中断来实现进程之间的切换。当然，我们的操作系统也支持进程主动放弃执行权。具体内容请阅读后续章节。

注解

读者可能会有疑问：既然只有一个处理器，即使采用上述方式，同一时间最多也还是只能运行一个应用，而且还需要占用更多内存来同时加载所有应用。那么这样做究竟有什么意义呢？

读者可以带着这个问题继续阅读。后续我们将介绍这种方式具体能够解决哪些实际问题。

实践体验

获取分时多任务系统的代码：

```
$ git checkout ch3
```

在 qemu 模拟器上运行本章代码：

```
$ make test BASE=1
```

多道程序的应用分别会输出一个不同的字母矩阵。当他们交替执行的时候，我们将看到字母行的交错输出：

另外需要说明的是一点是：与上一章不同，应用的编号不再决定其被加载运行的先后顺序，而仅仅能够改变应用被加载到内存中的位置。

本章代码树

```
uCore-Tutorial-Code-2025S-ch3/
├─ Makefile
├─ os/
│   ├─ console.c
│   ├─ console.h
│   ├─ const.h
│   ├─ defs.h
│   ├─ entry.S
│   ├─ kernel.ld
│   └─ kernelld.py
```

```
|   ├── loader.c
|   ├── loader.h
|   ├── log.h
|   ├── main.c
|   ├── pack.py
|   ├── printf.c
|   ├── printf.h
|   ├── proc.c
|   ├── proc.h
|   ├── riscv.h
|   ├── sbi.c
|   ├── sbi.h
|   ├── string.c
|   ├── string.h
|   ├── switch.s
|   ├── syscall.c
|   ├── syscall.h
|   ├── syscall_ids.h
|   ├── timer.c
|   ├── timer.h
|   ├── trampoline.s
|   ├── trap.c
|   ├── trap.h
|   └── types.h
├── README.md
├── scripts/
|   ├── kernelld.py
|   └── pack.py
└── user
```

本章代码导读

随着章节的推进，我们的OS从测例嵌入main.c到批处理的OS系统，再到本章的多进程OS，可以说我们的OS已经初具雏形了。在进入本章的内容之前，大家需要对进程有一个清晰的认识。进程就是“执行中的程序”，因此每一个进程需要有程序运行所需要的资源。每一个进程又有着自己的优先级，使得进程在调度的时候存在某种机制使得高优先级的进程能够优先执行，而低优先级的进程又不能永远不执行。我们本章就要设计一个支持优先级进程调度的OS。

同时，进程的引入也意味着同时存在多个app在运行，这意味着我们需要一次将多个测例的bin文件移动到指定的内存位置之中，并且为每一个bin都做好配套的初始化。

多道程序放置与加载

本节导读

在本章的引言中我们提到每个应用都需要按照它的编号被分别放置并加载到内存中不同的位置。本节我们就来介绍它是如何实现的。通过具体实现，可以看到多个应用程序被一次性地加载到内存中，这样在切换到另外一个应用程序执行会很快，不像前一章介绍的操作系统，还要有清空前一个应用，然后加载当前应用的过程与开销。

但我们也会了解到，每个应用程序需要知道自己运行时在内存中的不同位置，这对应用程序的编写带来了一定的麻烦。而且操作系统也要知道每个应用程序运行时的位置，不能任意移动应用程序所在的内存空间，即不能在运行时根据内存空间的动态空闲情况，把应用程序调整到合适的空闲空间中。

多道程序的放置

我们仍然使用pack.py以及kernel.py来生成链接测例bin文件的link_app.S以及kernel_app.ld两个文件。这些内容相较第二章并没有任何改变。主要改变的是对多道程序的加载上，要求同时加载多个程序了。

多道程序加载

从本章开始不再使用上一章批处理操作系统的run_next_app函数。让我们看看loader.c文件之中修改了什么。

```
// os/loader.c

int load_app(int n, uint64* info) {
    uint64 start = info[n], end = info[n+1], length = end - start;
    memset((void*)BASE_ADDRESS + n * MAX_APP_SIZE, 0, MAX_APP_SIZE);
    memmove((void*)BASE_ADDRESS + n * MAX_APP_SIZE, (void*)start, length);
    return length;
}

// load all apps and init the corresponding `proc` structure.
// 这个函数的更多细节在之后讲解
int run_all_app()
{
    for (int i = 0; i < app_num; ++i) {
        struct proc *p = allocproc();
        struct trapframe *trapframe = p->trapframe;
        load_app(i, app_info_ptr);
        uint64 entry = BASE_ADDRESS + i * MAX_APP_SIZE;
        trapframe->epc = entry;
        trapframe->sp = (uint64)p->ustack + USER_STACK_SIZE;
        p->state = RUNNABLE;
    }
    return 0;
}
```

可以看到，进程 load 的逻辑其实没有变化。但是我们在 run_all_app 函数之中一次性加载了所有的测例程序。具体的方式是遍历每一个app获取其放置的位置，并根据其序号 i 设置相对于 BASE_ADDRESS 的偏移量作为程序的起始位置。我们认为设定每个进程所使用的空间是 `[0x80400000 + i*0x20000, 0x80400000 + (i+1)*0x20000)`，每个进程的最大 size 为 `0x20000`，`i` 即为进程编号。需要注意此时同时完成了每一个程序对应的进程的初始化以及状态的设置（对进程还不熟悉的同学可以阅读下一节）。

现在应用程序加载完毕了。不同于批处理操作系统，我们该如何执行它们呢？

进程基础结构

本节导读

本节会介绍进程的调度方式。这是本章的重点之一。

进程的概念

导语中提到了，进程就是运行的程序。既然是程序，那么它就需要程序执行的一切资源，包括栈、寄存器等等。不同于用户线程，用户进程有着自己独立的用户栈和内核栈。但是无论如何寄存器是只有一套的，因此进程切换时对于寄存器的保存以及恢复是我们需要关心的问题。

为了研究进程的切换，我们先来搞懂用户进程长啥样，是如何运行的。不妨从上一节的 `run_all_app` 函数开始研究：

```
int run_all_app()
{
    for (int i = 0; i < app_num; ++i) {
        struct proc *p = allocproc();
        struct trapframe *trapframe = p->trapframe;
        load_app(i, app_info_ptr);
        uint64 entry = BASE_ADDRESS + i * MAX_APP_SIZE;
        trapframe->epc = entry;
        trapframe->sp = (uint64)p->ustack + USER_STACK_SIZE;
        p->state = RUNNABLE;
    }
    return 0;
}
```

首先介绍 `struct proc` 的定义。本章中新增的proc.h定义了我们OS的进程的PCB（进程管理块，和进程——对应。它包含了进程几乎所有的信息）结构体；

struct（结构体）是C语言中一种自定义数据类型，它允许将多个不同类型的数据项组合成一个整体。

struct将相关的数据项打包在一起，形成一个逻辑整体，代码更清晰、更易维护。

struct可以创建面向对象风格的数据抽象

struct是C语言中组织复杂数据的基石，在操作系统开发中尤为重要。

```
// os/proc.h

struct proc {
    enum procstate state; // 进程状态
    int pid;              // 进程ID
    uint64 ustack;        // 进程用户栈虚拟地址(用户页表)
    uint64 kstack;        // 进程内核栈虚拟地址(内核页表)
    struct trapframe *trapframe; // 进程中断帧
    struct context context; // 用于保存进程内核态的寄存器信息，进程切换时使用
};

enum procstate {
    UNUSED, // 未初始化
    USED,   // 基本初始化，未加载用户程序
    SLEEPING, // 休眠状态(未使用，留待后续拓展)
    RUNNABLE, // 可运行
    RUNNING, // 当前正在运行
}
```

```
    ZOMBIE,    // 已经 exit
};
```

可以看到每一个进程的PCB都保存了它当前的状态以及它的PID（每个进程的PID不同）。同时记录了其用户栈和内核栈的起始地址。`trapframe` 和 `context` 在异常中断的切换以及进程之间的切换起到了保存的重要作用。

进程的状态是大家比较熟悉的问题了。OS课程上将进程的状态分为创建、就绪、执行、等待以及结束5大阶段（未来还会有挂起）。在我们的OS之中对状态的分类略有不同。我们一般用 `RUNNABLE` 代表就绪的进程，`RUNNING` 代表正在执行的进程，`UNUSED` 代表池中未分配或已经结束的进程，`USED` 代表已经分配好但是还未加载完毕的进程。

进程的基本管理

在我们的OS之中，我们采用了非常朴素的进程池方式来存放进程：

```
// os/trap.c

struct proc pool[NPROC];    // 全局进程池
struct proc idle;           // boot 进程
struct proc* current_proc;  // 指示当前进程

// 由于还有没内存管理机制，静态分配一些进程资源
char kstack[NPROC][PAGE_SIZE];
__attribute__((aligned(4096))) char ustack[NPROC][PAGE_SIZE];
__attribute__((aligned(4096))) char trapframe[NPROC][PAGE_SIZE];
```

可以看到我们最多同时有 `NPROC` 个进程，每一个进程的用户栈、内核栈以及`trapframe`所需的空間已经预先分配好了。当然缺点是进程池空间有限，不过直到lab8之前大家都无需担心这个问题。

这里的 `idle` 进程是我们的 `boot` 进程，是我们执行初始化的进程，事实上，在引入用户进程前，`idle` 是唯一一个进程。比较重要的是 `current_proc`，它代表着当前正在执行的进程。因此这个变量在进程切换时也需要维护来保证其正确性。活用此变量能大大方便我们的编程。

进程模块初始化函数如下：

```
// kernel/trap.c

void procinit()
{
    struct proc *p;
    for(p = pool; p < &pool[NPROC]; p++) {
        p->state = UNUSED;
        p->kstack = (uint64)kstack[p - pool];
        p->ustack = (uint64)ustack[p - pool];
        p->trapframe = (struct trapframe*)trapframe[p - pool];
    }
    idle.kstack = (uint64)boot_stack_top;
    idle.pid = 0;
}
```

进程的分配

回到 `run_all_app` 函数，可以注意到首每个用户进程都被分配了一个 `proc` 结构，通过 `alloc_proc` 函数。进程的分配实际上本质就是从进程池中挑选一个还未使用（状态为 `UNUSED`）的位置分配给进程。具体代码如下：

```
// os/proc.c

// Look in the process table for an UNUSED proc.
// If found, initialize state required to run in the kernel.
// If there are no free procs, or a memory allocation fails, return 0.
struct proc *allocproc()
{
    struct proc *p;
    for (p = pool; p < &pool[NPROC]; p++) {
        if (p->state == UNUSED) {
            goto found;
        }
    }
    return 0;

found:
    p->pid = allocpid();
    p->state = USED;
    memset(&p->context, 0, sizeof(p->context));
    memset(p->trapframe, 0, PAGE_SIZE);
    memset((void *)p->kstack, 0, PAGE_SIZE);
    p->context.ra = (uint64)usertrapret;
    p->context.sp = p->kstack + PAGE_SIZE;
    return p;
}
```

分配进程需要初始化其PID以及清空其栈空间，并设置 `context` 第一次运行的入口地址 `usertrapret`，使得进程能够从内核的S态返回U态并执行自己的代码。我们需要看看进程切换相关的东西了。

多道程序与协作式调度

本节导读

上一节我们介绍了任务管理的基本结构。本节我们将具体讲解进程切换是如何实现的。

多道程序背景与 `yield` 系统调用

还记得第二章介绍的批处理系统吗？它的出现是因为人们发现 CPU 并没有一直执行应用程序。在一个程序运行结束，到操作员手动更换下一个程序卡片的这段时间里，CPU 是空闲的。这种空闲浪费了宝贵的计算资源。

批处理系统的出现解决了这个问题。它能够自动加载和运行程序，并把一些简单任务交给外围设备处理，让 CPU 更专注于计算，从而提高了 CPU 的使用率。

不过，CPU 的使用率还能进一步提高。随着程序变得越来越复杂，它们经常需要访问外部设备（比如键盘、磁盘、显示器等），这些设备统称为输入/输出设备。CPU 向设备发出请求后，需要等待设备处理完毕才能拿到结果。

例如，从磁盘读取数据时，CPU 会告诉磁盘要读取哪个位置的数据，以及数据应该放到内存的哪里。然后 CPU 就必须等待磁盘找到数据并完成传输。

那么 CPU 如何知道设备是否处理完毕呢？通常，设备会提供一个状态寄存器。CPU 需要反复读取这个寄存器的值，直到状态显示“完成”才能继续执行后面的操作。

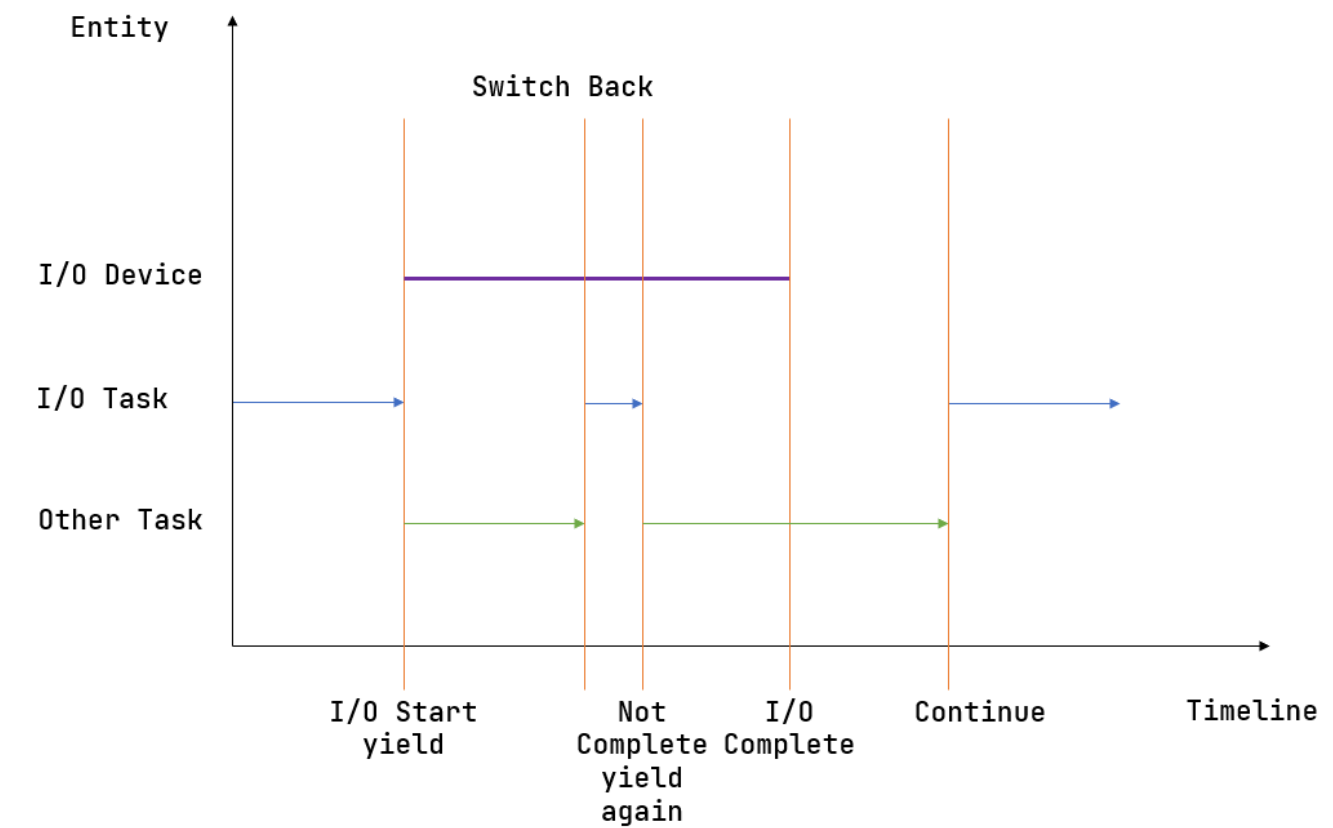
问题是，外部设备的处理速度比 CPU 慢得多。在等待期间，CPU 只是在“空转”，什么有用的事也没做，这又造成了 CPU 资源的浪费。

注：这里我们假设 CPU 只能通过主动读取设备寄存器来获取状态。

多道程序的思想可以解决这个问题：让内核同时管理多个应用程序。当一个程序等待设备时，内核可以切换到另一个程序去执行。过一段时间再切换回来，检查设备是否就绪。如果就绪，就继续执行原来的程序；如果还没就绪，就再次切换走。

这样一来，只要系统中同时有足够多的程序，CPU 就几乎可以一直保持工作状态，而不用空等设备。这种任务切换是通过一个叫做 `sys_yield` 的系统调用来触发的。应用程序调用这个系统调用，表示自己主动让出 CPU 的使用权。

这就是“协作式”调度中“协作”的含义。每个程序都会一直运行，直到它主动调用 `sys_yield` 交出 CPU。内核将让出 CPU 的权力交给了应用程序本身，依靠所有应用程序互相协作，共同实现最大化利用 CPU 资源的目标。在计算机发展早期，程序比较简单，程序员也比较遵守规则，这种基于信任的协作方式是可行的。



上图展示了多道程序执行的典型过程。横轴是时间，纵轴表示正在执行的实体。

1. 一开始，一个应用程序（蓝色）向设备发出请求，设备（紫色）开始工作。
2. 设备需要工作很长时间，所以蓝色程序不会傻等，而是调用 `sys_yield` 主动让出 CPU。
3. 内核切换到另一个程序（绿色）执行。
4. 绿色程序执行一段时间后，也调用了 `sys_yield`。
5. 内核切换回蓝色程序。蓝色程序检查设备状态，发现请求还没处理完，于是再次调用 `sys_yield`。

6. 内核再次切换到绿色程序执行。
7. 绿色程序执行后再次让出 CPU。
8. 这次切换回蓝色程序时，它发现设备已经处理完请求了，于是可以继续执行后续操作。

上面我们是从“避免等待设备”的角度引入 `sys_yield` 的。但实际上，调用 `sys_yield` 不一定是为了等设备。随着内核功能的增加，我们会遇到各种需要等待的事件，都可以通过立即调用 `sys_yield` 来避免 CPU 空等。

思考：`sys_yield` 的缺点

请你想一想，`sys_yield` 这种方式有什么缺点？

当一个程序调用 `sys_yield` 交出 CPU 后，它下一次什么时候能再被调度执行，取决于内核的调度策略和当前所有程序的运行情况。这个时间可能远晚于该程序所等待的事件（比如设备处理完成）实际发生的时间。

这会导致程序的响应时间不稳定，有时可能非常长。试想一下，如果你敲下键盘后，要等几分钟才能在屏幕上看到字符，这显然是无法接受的。

不过别担心，我们后面会介绍更优雅、更高效的解决方案。

我们给出 `sys_yield` 的标准接口：

第三章新增系统调用（一）

```
/// 功能：应用主动交出 CPU 所有权并切换到其他应用。
/// 返回值：总是返回 0。
/// syscall ID: 124
int sys_yield();
```

然后是用户库对应的实现和封装：

```
// user/lib/syscall.c

int sched_yield()
{
    return syscall(SYS_sched_yield);
}
```

进程切换

接下来，我们将介绍内核应如何实现该系统调用。

进程的切换

下面我们将讲解本章最关键的知识点——进程的切换（也称为进程调度）。

什么是进程的切换？

谈到“切换”，相信大家会对第二章中异常触发后从用户态（U态）切换到内核态（S态）的过程记忆犹新。实际上，进程切换的过程与它非常相似。

注释：这里“进程切换”指的是操作系统将 CPU 的执行权从一个进程转移到另一个进程的过程。类似于你在做两件事时，先保存第一件事的进度，然后去做第二件事，做完后再回来继续第一件事。

我们可以这样类比理解：在第二章中，流程是进程1 → 内核态处理异常 → 返回进程1。那么，我们完全可以将这个流程扩展为进程1 → 内核进行切换 → 进程2 → 内核再次切换 → 返回进程1，从而实现执行流的切换，并保证过程中各进程的状态被正确保存和恢复。

当然，直接这样做会显得比较复杂。在实现上，我们采用一种更传统的方式，但核心思路是相同的。

请大家回忆一下进程 PCB 结构体中的两个成员，它们是实现进程切换的关键：

注释：PCB（Process Control Block）即进程控制块，是操作系统为每个进程维护的数据结构，用来记录进程的状态、上下文等信息。在切换时，我们需要保存当前进程的“现场”，并恢复下一个进程的“现场”。

```
struct trapframe *trapframe;
struct context context;
```

`trapframe` 大家在第二章已经和它打过交到了。那么 `context` 这个结构体又记录了什么呢？

```
// os/trap.h

// Saved registers for kernel context switches.
struct context {
    uint64 ra;
    uint64 sp;
    // callee-saved
    uint64 s0;
    uint64 s1;
    uint64 s2;
    uint64 s3;
    uint64 s4;
    uint64 s5;
    uint64 s6;
    uint64 s7;
    uint64 s8;
    uint64 s9;
    uint64 s10;
    uint64 s11;
};
```

它相比 `trapframe`，只记录了寄存器的信息。聪明的你可能已经发现它们都是被调用者保存的寄存器。在切换的核心函数 `swtch`（注意拼写）之中，就是对这个结构体进行了操作：

```
# Context switch
#
# void swtch(struct context *old, struct context *new);
#
# Save current registers in old. Load from new.

.global swtch

# a0 = &old_context, a1 = &new_context

swtch:
    sd ra, 0(a0)          # save `ra`
```

```

sd sp, 8(a0)          # save `sp`
sd s0, 16(a0)
sd s1, 24(a0)
sd s2, 32(a0)
sd s3, 40(a0)
sd s4, 48(a0)
sd s5, 56(a0)
sd s6, 64(a0)
sd s7, 72(a0)
sd s8, 80(a0)
sd s9, 88(a0)
sd s10, 96(a0)
sd s11, 104(a0)

ld ra, 0(a1)          # restore `ra`
ld sp, 8(a1)          # restore `sp`
ld s0, 16(a1)
ld s1, 24(a1)
ld s2, 32(a1)
ld s3, 40(a1)
ld s4, 48(a1)
ld s5, 56(a1)
ld s6, 64(a1)
ld s7, 72(a1)
ld s8, 80(a1)
ld s9, 88(a1)
ld s10, 96(a1)
ld s11, 104(a1)

ret                  # return to new `ra`

```

为什么只切换这些寄存器就能实现一个切换的效果呢？这是因为执行了 `swtch` 切换状态之后，切换的目标进程恢复了保存在 `context` 之中的寄存器，并且 `sp` 寄存器也指向了它自己栈的位置，`ra` 指向自己测例代码的位置而不是之前函数的位置，这已经足够其从切换出去的位置继续执行了（切换的过程可以视为一次函数调用）。因为真正切换 `swtch` 都在内核态发生，也无需记录更多的数据。

总结一下，`swtch` 函数干了这些事情：

- 执行流：通过切换 `ra`
- 堆栈：通过切换 `sp`
- 寄存器：通过保存和恢复被调用者保存寄存器。调用者保存寄存器由编译器生成的代码负责保存和恢复。

一旦你理解了上述的过程，那么本章剩余内容就会十分简单~~

下面介绍进程切换的具体细节。

idle进程与scheduler

大家可能注意到 `proc.c` 文件中除了 `current_proc` 还记录了一个 `idle_proc`。这个进程是干什么的呢？实际上，`idle` 进程是第一个进程(boot进程)，也是唯一一个永远会存在的进程，它还有一个大家更熟悉的面孔，它就是 `os` 的 `main` 函数。

是时候从头开始梳理从机器 boot 到多个用户进程相互切换到底发生了什么了。

```

void main() {
    clean_bss();    // 清空 bss 段
    trapinit();    // 开启中断
    batchinit();    // 初始化 app_info_ptr 指针
    procinit();    // 初始化线程池
    // timerinit();    // 开启时钟中断，现在还没有
    run_all_app();  // 加载所有用户程序
    scheduler();    // 开始调度
}

```

可以看到，在 `main` 函数完成了一系列的初始化，并且执行了 `run_all_app` 加载完了所有测例之后。它就进入了 `scheduler` 调度函数。这个函数就完成了一系列的调度工作：

```

void
scheduler(void)
{
    struct proc *p;

    for(;;){
        for(p = pool; p < &pool[NPROC]; p++) {
            if(p->state == RUNNABLE) {
                p->state = RUNNING;
                current_proc = p;
                swtch(&idle.context, &p->context);
            }
        }
    }
}

```

可以看到一旦 `main` 进入调度状态就进入一个死循环再也回不去了。。但它也没必要回去，它现在活着的意义就是为了进行进程的调度。在循环中每一次 `idle` 进程都会遍历整个进程池来寻找 `RUNNABLE`（就绪）状态的进程并执行 `swtch` 函数切换到它。我们这里的 `scheduler` 函数就是最普通的调度函数，完全没有考虑优先度以及复杂度。

这里大家要思考一下，这个函数写对了吗？它真的满足我们每次执行遍历一次的要求，而不是写成了每次都从第0个进程开始遍历查找吗？

yield函数的实现

`yield` 函数具体实现如下：

```

// os/proc.c

// Give up the CPU for one scheduling round.
void yield(void)
{
    current_proc->state = RUNNABLE;
    sched();
}

void sched(void)
{

```

```
struct proc *p = curr_proc();
swtch(&p->context, &idle.context);
}
```

它本质就是主动放弃执行，并把 `context` 移交给负责 scheduler 进程的 `idle` 进程。那这个时候 `idle` 进程在干什么？

`idle` 线程死循环在了一件事情上：寻找一个 `RUNNABLE` 的进程，然后切换到它开始执行。当这个进程调用 `sched` 后，执行流会回到 `idle` 线程，然后继续开始寻找，如此往复。直到所有进程执行完毕，在 `sys_exit` 系统调用中有统计计数，一旦 `exit` 的进程达到用户程序数量就关机。

也就是说，所有进程间切换都需要通过 `idle` 中转一下。那么可不可以一步到位呢？答案是肯定的，其实 [rust版代码lab3](#) 就是采取这种实现：在一个进程退出时，直接寻找下一个就绪进程，然后直接切换过去，没有 `idle` 的中转。两种实现都是可行的。

在了解这些之后，我们就可以实现协作式调度了，主要是 `sys_yield` 系统调用，其实现十分简单，请同学们自行查看 `kernel/syscall.c`。

分时多任务系统与抢占式调度

本节导读

本节将重点讲解操作系统如何处理中断，以及如何实现对应用程序的抢占。为此，我们将“任务”这个概念进一步延伸和细化。

1. 分时多任务：操作系统会管理每一个应用程序，让它们以“时间片”为单位，轮流占用处理器运行。
2. 时间片轮转调度：当一个程序用完分配给它的时间片后，操作系统会强制暂停（抢占）这个程序，然后调用队列中的下一个程序执行。这个过程循环往复，从而在任务级别上实现了基于时间片的轮流调度。

分时多任务系统的背景

在上一节，我们介绍了“多道程序”的设计。这种设计允许一个应用程序在等待外部设备（比如读取磁盘）时，主动让出CPU，切换到其他应用程序运行，目的是让CPU总体利用率达到最高。在那个背景下，应用程序本身并不太关心自己的运行状态。即使它在主动让出（`yield`）CPU之后，需要等待很久才能再次获得CPU，导致它真正在CPU上执行的两个时间段间隔很长，应用程序也无所谓。因为它们的共同目标是追求CPU的总体利用率最高，这也可以等价为一个叫做吞吐量（Throughput）的指标。

通俗地说，吞吐量可以理解为：在某个时间点启动一组应用程序，要求在一段固定的时间内，能够执行完成的应用数量最多，或者所有应用完成的总进度百分比最大。因此，当时的应用程序和程序员都默认一个共识：只要CPU一直在干“实实在在的活”，就是好的。

从今天的角度来看，当时的应用程序更像一种后台应用。我们把它放入执行队列后，只需要定期检查一下它的运行状态即可。但随着技术发展，出现了越来越多的交互式应用（比如图形界面程序、游戏）。这类应用的一个重要目标是提升用户操作的响应速度，从而优化使用体验。对于这类应用来说，即使需要等待外部设备或某些事件，它们通常也不愿意主动让出（`yield`）CPU使用权，因为这样做可能会导致用户感受到无法接受的延迟。这意味着，应用程序之间从互相合作，更多地转向了互相竞争宝贵的硬件资源。

如果应用程序自己很少主动让出CPU，那么操作系统（内核）就必须收回之前下放的权力，由它自己来集中管理CPU资源，并合理地分配给各个应用。这就是内核需要提供的任务调度能力。

我们可以把多道程序的调度机制归类为协作式调度。它的特点是：只要一个应用不主动 `yield` 交出CPU，它就会一直执行下去。相反，抢占式调度则意味着应用程序随时都可能被内核强制切换出去。

现代操作系统的任务调度算法基本都是抢占式的。它要求每个应用只能连续执行一段时间，然后内核就会强制把它切换出去。通常，我们用时间片来衡量一个应用可以连续执行的时长，每个时间片可能只有几毫秒。调度算法需要考虑两个核心问题：

- 1. 每次切换前，给一个应用分配多少个时间片去执行？
- 2. 接下来要切换到哪个应用去执行？

评价一个调度算法，主要看性能和公平性两个维度。公平性要求多个应用分到的时间片比例不能相差太大。

时间片轮转调度

为了简单起见，在本书中，我们将使用时间片轮转调度算法来对应用程序进行调度。这个算法稍加扩展就能满足我们后续的所有需求。本章我们只需要最基础的RR算法。

用文字描述基础的RR算法：内核维护一个应用程序的任务队列。每次从队列头部取出一个应用，让它执行一个时间片。时间片用完后，内核将这个应用放到队列尾部，然后再从队列头部取出下一个应用执行。如此循环，直到所有应用执行完毕。

RISC-V 架构中的中断

实现时间片轮转调度的核心在于计时。操作系统的计时功能，依赖于硬件提供的时钟中断。在介绍时钟中断之前，我们先简单了解一下什么是中断。

中断和我们在第二章介绍的、用于系统调用的陷入，都属于“异常”。但它们被触发的原因是不同的。对于单个处理器核心来说：

陷入 与导致陷入的那条指令的执行是同步的。我们总能找到一条具体的指令，是它导致了这次陷入。
中断则异步于处理器当前正在执行的指令。中断来自哪个外部设备、何时触发，与处理器当前执行的指令没有直接关系。

从底层硬件的角度区分同步和异步
或许从硬件层面能更好理解同步和异步。想象一个经典的五级流水线处理器，包含取指令、解码、运算、访问内存、写回寄存器等单元，这些都属于执行指令所需的硬件。如果某条指令执行出了问题，一定会被其中某个单元发现，并报告给流水线控制单元。于是，处理器会在执行下一条指令前，先进入异常处理流程。也就是说，异常是在这些执行单元内部被察觉和处理的。
而对于中断，可以想象是另一套完全独立的电路（比如时钟中断，就是一个简单的计数和比较电路）想要通知处理器。这套电路只用一根导线连接到处理器核心。当它想触发中断时，就给这根导线一个高电平或一个上升沿信号。处理器会在每执行完一条指令后，去检查一下这根线的状态，然后决定是继续执行下条指令，还是进入中断处理流程。也就是说，在大多数时候，执行指令的硬件单元和可能发起中断的电路是独立、并行运行的，它们之间只有一根信号线相连。除此之外，执行指令的单元完全不知道对方电路处于什么状态。

在不考虑其他指令集扩展的情况下，RISC-V 架构定义了以下几种中断：

RISC-V 中断一览表

中断类型 (Interrupt)	异常码 (Exception Code)	描述 (Description)
1	1	监管者模式软件中断 (Supervisor software interrupt)
1	3	机器模式软件中断 (Machine software interrupt)
1	5	监管者模式时钟中断 (Supervisor timer interrupt)

中断类型 (Interrupt)	异常码 (Exception Code)	描述 (Description)
1	7	机器模式时钟中断 (Machine timer interrupt)
1	9	监管者模式外部中断 (Supervisor external interrupt)
1	11	机器模式外部中断 (Machine external interrupt)

RISC-V的中断可以分为三大类：

- 1. 软件中断
- 2. 时钟中断
- 3. 外部中断

另外，与异常相比，中断和CPU的特权级关系更紧密。可以看到，上面每一类中断都有M（机器）模式和S（监管者）模式两个版本。中断的特权级决定了它是否会被屏蔽，以及需要陷入到CPU的哪个特权级进行处理。

判断一个中断是否会被屏蔽，遵循以下规则：

- 1. 如果中断的特权级低于CPU当前运行的特权级，则该中断被屏蔽，不会被处理。
- 2. 如果中断的特权级高于或等于CPU当前的特权级，则需要通过特定的CSR（控制和状态寄存器）来进一步判断它是否被屏蔽。

以操作系统内核所在的S特权级为例，与中断屏蔽相关的CSR是 `sstatus` 和 `sie`。

- 1. `sstatus` 寄存器中的 `sie` 位是S特权级的总中断开关。如果将它清零，所有S级中断都会被屏蔽。
- 2. 即使 `sstatus.sie` 位为1（打开总开关），还要看 `sie` 这个专门的CSR。它有三个关键位：`ssie`（软件中断使能）、`stie`（时钟中断使能）、`seie`（外部中断使能），分别控制三类S级中断的具体开关。

例如，对于一个S级的时钟中断，如果CPU当前运行在不高于S的特权级（即S或U级），那么需要 `sstatus.sie` 和 `sie.stie` 同时为1，这个中断才不会被屏蔽。如果CPU当前运行在高于S的特权级（即M级），那么这个中断无论如何都会被屏蔽。

如果一个中断没有被屏蔽，接下来就需要通过“陷入”来进行处理。具体会陷入到哪个特权级，与一些“中断代理”CSR的设置有关。默认情况下，所有的中断都需要陷入到最高特权级M模式处理。但通过设置这些代理CSR，可以让中断陷入到较低的特权级（如S模式）处理，不过，陷入到的目标特权级不能低于该中断本身的特权级。

事实上，所有的异常默认也都是陷入到M模式处理的，它们也有一套对应的“异常代理”CSR，设置后也可以交给低特权级处理。

我们会在附录C：深入机器模式：RustSBI中更详细介绍中断/异常代理。在正文部分，我们只需要了解以下两点即可：

- 1. 包括系统调用（即来自U特权级的“环境调用”指令）在内的所有异常，都会陷入到S特权级处理。
- 2. 我们只需要关心S特权级的时钟、软件和外部中断，并且它们都会被陷入到S特权级处理。

默认情况下，当CPU因为陷入而进入某个特权级后，在处理这个陷入的过程中，同特权级的中断都会被自动屏蔽。这里我们需要对第二章介绍的陷入发生时的硬件机制做一点补充，以陷入到S特权级为例：

- 1. 当陷入发生时，硬件会自动将 `sstatus.sie` 的当前值保存到 `sstatus.spie` 字段中，然后将 `sstatus.sie` 置为0。这样就屏蔽了所有S特权级的中断。
- 2. 当陷入处理完毕，执行 `sret` 指令返回时，硬件会将 `sstatus.spie` 的值恢复回 `sstatus.sie`。

也就是说，如果不去手动修改 `sstatus` CSR，在只考虑S级中断的情况下，是不会出现嵌套中断的。嵌套中断是指，在处理一个中断的过程中，又再次触发了新的中断，从而导致又一次陷入。

嵌套中断与嵌套陷入

嵌套中断可以分成两种情况：1) 在处理一个中断时，被同特权级的中断打断；2) 被更高特权级的中断打断。默认的硬件机制会防止第一种情况发生（因为自动关了同级中断）。但第二种情况，无论怎么设置都是可能发生的，因为高特权级中断无法被低特权级屏蔽。

嵌套陷入的范围更广，它指在处理任何一个陷入（可以是中断，也可以是异常）的过程中，又发生了新的陷入。嵌套中断是嵌套陷入的一种特定情况。

RISC-V 架构的 U 特权级中断

目前，RISC-V 的用户态（U特权级）中断功能是作为一个代号为“N”的指令集扩展而存在的。对此有兴趣的读者，可以查阅最新版的 RISC-V 特权级架构规范来了解更多细节。

时钟中断与计时器

由于需要一种计时机制，RISC-V 架构要求处理器要有一个内置时钟，其频率一般低于 CPU 主频。此外，还有一个计数器统计处理器自上电以来经过了多少个内置时钟的时钟周期。在 RV64 架构上，该计数器保存在一个 64 位的 CSR `mtime` 中，我们无需担心它的溢出问题，在内核运行全程可以认为它是一直递增的。

另外一个 64 位的 CSR `mtimecmp` 的作用是：一旦计数器 `mtime` 的值超过了 `mtimecmp`，就会触发一次时钟中断。这使得我们可以方便的通过设置 `mtimecmp` 的值来决定下一次时钟中断何时触发。

可惜的是，它们都是 M 特权级的 CSR，而我们的内核处在 S 特权级，是不被硬件允许直接访问它们的。好在运行在 M 特权级的 SEE 已经预留了相应的接口，我们可以调用它们来间接实现计时器的控制：

```
// os/timer.c

/// read the `mtime` regiser
uint64 get_cycle()
{
    return r_time();
}
```

`timer.c` 子模块的 `get_cycle` 函数可以取得当前 `mtime` 计数器的值；

```
// os/sbi.c

void set_timer(uint64 stime)
{
    sbi_call(SBI_SET_TIMER, stime, 0, 0);
}

// os/timer.c

#define TICKS_PER_SEC (100)
#define CPU_FREQ (12500000)

/// Set the next timer interrupt
void set_next_timer()
{
    const uint64 timebase = CPU_FREQ / TICKS_PER_SEC;
```



```
    set_timer(get_cycle() + timebase);  
}
```

sbi.c 有一个 `set_timer` 调用，是一个由 SEE 提供的标准 SBI 接口函数，它可以用来设置 `mtimecmp` 的值。

timer.c 的 `set_next_trigger` 函数对 `set_timer` 进行了封装，它首先读取当前 `mtime` 的值，然后计算出 10ms(也就是一个tick) 之内计数器的增量，再将 `mtimecmp` 设置为二者的和。这样，10ms 之后一个 S 特权级时钟中断就会被触发。

至于增量的计算方式，`CLOCK_FREQ` 是一个预先获取到的各平台不同的时钟频率，单位为赫兹，也就是一秒钟之内计数器的增量，10ms 的话只需除以常数 `TICKS_PER_SEC` 也就是 100 即可。

syscall 实现

既然引入了 time 模块，我们也顺势实现了 time 有关的一个系统调用 `gettimeofday`：

```
typedef struct {  
    uint64 sec; // 自 Unix 纪元起的秒数  
    uint64 usec; // 微秒数，也就是秒的小数点后的内容  
} TimeVal;  
  
/// tz 参数表示时区，这里我们忽略这个参数  
uint64 sys_gettimeofday(TimeVal *val, int _tz)  
{  
    uint64 cycle = get_cycle();  
    val->sec = cycle / CPU_FREQ;  
    val->usec = (cycle % CPU_FREQ) * 1000000 / CPU_FREQ;  
    return 0;  
}
```

我们只需要把 `get_cycle` 得到的 cycle 数换算了秒和微妙，填入对应结构即可。

syscall 标准定义如下：

第三章新增系统调用（二）

```
/// 功能：获取当前的时间，填写 TimeVal 结构体  
/// 返回值：始终为 0  
/// syscall ID: 169  
int sys_gettimeofday(TimeVal *val, int tz)
```

抢占式调度

有了时钟中断和计时器，抢占式调度就很容易实现了：

```

void usertrap() {
    // ...
    switch(cause) {
    case SupervisorTimer:
        set_next_timer();
        yield();
        break;
    // ...
    }
}

```

我们只需在 `usertrap` 函数下新增一个分支，当发现触发了一个 S 特权级时钟中断的时候，首先重新设置一个 10ms 的计时器，然后调用上一小节提到的 `yield` 函数暂停当前应用并切换到下一个。

为了避免 S 特权级时钟中断被屏蔽，我们需要在执行第一个应用之前进行一些初始化设置：

```

// os/main.c

void main{
    // ...
    timer_init();
    // ...
}

// os/trap.c

/// Enable timer interrupt
void timer_init()
{
    // 设置 ``sie.stie`` 使得 S 特权级时钟中断不会被屏蔽；
    w_sie(r_sie() | SIE_STIE);
    // 设置第一个 10ms 的计时器。
    set_next_timer();
}

```

这样，当一个应用运行了 10ms 之后，一个 S 特权级时钟中断就会被触发。由于应用运行在 U 特权级，且 `sie` 寄存器被正确设置，该中断不会被屏蔽，而是 Trap 到 S 特权级内的我们的 `trap_handler` 里面进行处理，并顺利切换到下一个应用。这便是我们所期望的抢占式调度机制。

目前在等待某些事件的时候仍然需要 `yield`，其中一个原因是为了节约 CPU 计算资源，另一个原因是当事件依赖于其他的应用的时候，由于只有一个 CPU，当前应用的等待可能永远不会结束。这种情况下需要先将它切换出去，使得其他的应用到达它所期待的状态并满足事件的生成条件，再切换回来。

这里我们先通过 `yield` 来优化轮询(Busy Loop) 过程带来的 CPU 资源浪费。在 `ch3b_sleep.c` 这个应用中：

```
// user/src/ch3b_sleep.c

int main()
{
    // get_mtime() 是 gettimeofday 的封装，得到毫秒数。
    int64 current_time = get_mtime();
    int64 wait_for = current_time + 3000;
    while (get_mtime() < wait_for) {
        sched_yield();
    }
    puts("Test sleep OK!");
    return 0;
}
```

它的功能是等待 3000ms 然后退出。可以看出，我们会在循环里面 sched_yield() 来主动交出 CPU 而不是无意义的忙等。尽管我们不这样做，已有的抢占式调度还是会在它循环 10ms 之后切换到其他应用，但是这样能让内核给其他应用分配更多的 CPU 资源并让它们更早运行结束。

编程作业：系统调用追踪 (sys_trace)

在掌握了多道程序与分时多任务的基本原理后，本节将通过一个实际的编程作业来加深对系统调用机制和进程管理的理解。

作业目标

实现 sys_trace 系统调用（调用号 410），支持以下三种功能：

trace_request	功能	参数说明	返回值
0	读取用户内存	id = 虚拟地址	该地址的一个字节值，失败返回 -1
1	写入用户内存	id = 虚拟地址, data = 写入值	成功返回 0，失败返回 -1
2	查询系统调用次数	id = 系统调用号	该系统调用被当前进程调用的次数

问题分析

在开始编码之前，我们需要思考几个关键问题：

问题1：内核如何安全地访问用户空间的内存？

用户程序传递给内核的是**虚拟地址**，而内核需要访问的是**物理地址**。回顾本章介绍的页表机制，我们知道用户进程的虚拟地址需要经过页表转换才能得到物理地址。

查看 vm.c 中的地址转换函数：

```
// os/vm.c
uint64 useraddr(pagetable_t pagetable, uint64 va)
{
    uint64 page = walkaddr(pagetable, va);
    if (page == 0)
        return 0;
    return page | (va & 0xFFFULL); // 物理页基址 + 页内偏移
}
```

这个函数通过 `walkaddr` 查找虚拟地址对应的物理页，然后将物理页基址与页内偏移（低12位）组合，得到完整的物理地址。

问题2：系统调用次数应该存储在哪里？

由于每个进程需要独立统计各自的系统调用次数，这些计数必须存储在进程控制块（PCB）中。这样当进程切换时，每个进程都能维护自己的统计数据。

问题3：何时更新系统调用计数？

计数必须在**每次系统调用分发时**更新。查看系统调用入口函数 `syscall()`，在此处可以统一更新计数，无需修改每个具体的系统调用实现。

代码实现

第一步：扩展进程控制块

首先在 `proc.h` 中定义系统调用最大数量，并在 PCB 中添加计数数组：

```
// os/proc.h
#define MAX_SYSCALL_NUM (500) /* ch3: 系统调用最大数量 */

struct proc {
    // ... 原有字段 ...
    /* ch3: 系统调用计数数组 */
    int syscall_count[MAX_SYSCALL_NUM];
};
```

第二步：初始化计数数组

在进程分配函数 `allocproc()` 中初始化计数数组为零：

```
// os/proc.c
struct proc *allocproc()
{
    // ... 分配进程 ...

    /* ch3: 初始化系统调用计数 */
    memset(p->syscall_count, 0, sizeof(p->syscall_count));

    return p;
}
```

第三步：在系统调用分发器中更新计数

在 `syscall()` 函数中，每次处理系统调用时更新对应的计数：

```
// os/syscall.c
void syscall()
{
    struct trapframe *trapframe = curr_thread()->trapframe;
    int id = trapframe->a7; // RISC-V 约定：系统调用号在 a7 寄存器

    // ... 系统调用处理 ...

    /* ch3: 统计系统调用次数 */
    struct proc *p = curr_proc();
    if (id >= 0 && id < MAX_SYSCALL_NUM) {
        p->syscall_count[id]++;
    }
}
```

RISC-V 系统调用约定：调用号放在 `a7` 寄存器，参数放在 `a0-a5`，返回值放在 `a0`。

第四步：实现 `sys_trace`

```
// os/syscall.c

/* ch3: 系统调用追踪 */
int sys_trace(int trace_request, uint64 id, uint8 data)
{
    struct proc *p = curr_proc();

    if (trace_request == 0) {
        /* 读操作：从用户地址读取一个字节 */
        uint8 *addr = (uint8 *)useraddr(p->pagetable, id);
        if (addr == NULL)
            return -1;
        return *addr;
    } else if (trace_request == 1) {
        /* 写操作：向用户地址写入一个字节 */
        uint8 *addr = (uint8 *)useraddr(p->pagetable, id);
        if (addr == NULL)
            return -1;
        *addr = data;
        return 0;
    } else if (trace_request == 2) {
        /* 查询：返回指定系统调用的调用次数 */
        if (id >= MAX_SYSCALL_NUM)
            return -1;
        return p->syscall_count[id];
    }
    return -1;
}
```

第五步：注册系统调用

在 `syscall()` 的 `switch` 语句中添加入口：

```
// os/syscall.c - syscall() 函数
case SYS_trace:
    ret = sys_trace(args[0], args[1], args[2]);
    break;
```

测试验证

编译运行后，测试程序会验证以下功能：

- 1. **内存读写**：通过 `trace_request=0/1` 读写用户空间变量
- 2. **调用计数**：通过 `trace_request=2` 验证各系统调用的调用次数

知识点总结

通过本作业，我们深入理解了以下概念：

概念	说明
地址空间隔离	用户程序和内核使用不同的地址空间，内核访问用户内存需要地址转换
页表转换	虚拟地址通过页表转换为物理地址， <code>useraddr</code> 函数封装了这一过程
进程私有数据	每个进程有独立的 PCB，可存储进程特有的状态（如系统调用计数）
系统调用约定	RISC-V 的 <code>ecall</code> 指令触发系统调用，调用号在 <code>a7</code> ，参数在 <code>a0-a5</code>

进阶思考

在后续章节中，我们会发现 `sys_trace` 的简单实现存在安全隐患：

- 读操作应检查页面的 `PTE_R`（可读）权限位
- 写操作应检查页面的 `PTE_W`（可写）权限位

这些权限检查将在第四章（地址空间）的作业中完善。

修改文件清单

文件	修改内容
<code>os/proc.h</code>	添加 <code>MAX_SYSCALL_NUM</code> 宏和 <code>syscall_count</code> 数组
<code>os/proc.c</code>	在 <code>allocproc()</code> 中初始化计数数组
<code>os/syscall.c</code>	实现 <code>sys_trace()</code> ，在 <code>syscall()</code> 中更新计数